RAG Pipeline on the RBI Monetary Policy Report

A line-by-line technical walkthrough, the math and models behind each step, and interview defense notes

1. What This Project Actually Is

This is a Retrieval-Augmented Generation (RAG) system built over the RBI's April 2025 Monetary Policy Report (116 pages). Instead of asking an LLM a finance question and hoping it remembers the right number from training data, the system first searches the actual PDF for the most relevant passages, then hands only those passages to the LLM and forces it to answer strictly from them. The project also builds two layers of evaluation on top: classic information-retrieval metrics (Hit-Rate@k, MRR) that check whether the retriever found the right page, and DeepEval LLM-judge metrics (correctness, contextual relevancy, contextual recall) that check whether the final generated answer is actually good.

The pipeline has five real stages, and it's worth being able to name them cleanly in an interview:

- Ingest — load the PDF and split it into overlapping chunks
- Embed & Index — turn each chunk into a vector and store it in a vector database
- Retrieve — given a question, find the most similar chunks
- Rerank — re-score the retrieved chunks with a more accurate (but slower) model
- Generate & Evaluate — feed the top chunks to an LLM, and separately score both retrieval and generation quality

2. Step-by-Step Code Walkthrough

2.1 Library installation

```
!pip install -q langchain langchain-community langchain-groq \
    langchain-text-splitters chromadb sentence-transformers pypdf deepeval
```

What each library is actually for:

- langchain / langchain-community — orchestration glue. Provides standard interfaces (Document loaders, text splitters, vector store wrappers) so you can swap the embedding model, LLM, or vector DB without rewriting the pipeline.
- langchain-groq — the LangChain adapter for Groq's inference API (used to call Llama 3.1).
- chromadb — the vector database that stores chunk embeddings and does similarity search.
- sentence-transformers — the library behind both the embedding model (MiniLM) and the cross-encoder reranker.
- pypdf — the actual PDF text-extraction engine LangChain's PyPDFLoader calls under the hood.
- deepeval — the LLM-as-judge evaluation framework used later for correctness/relevancy/recall scoring.

Why this matters in an interview: if asked “why LangChain instead of calling the OpenAI/Groq API directly,” the honest answer is: LangChain isn't doing anything you couldn't hand-code, it's just standardizing the interface so

components are swappable and the code is shorter. The trade-off is an extra abstraction layer that can obscure what's actually happening — which is exactly why this report exists.

2.2 Loading and chunking the PDF

```
docs = PyPDFLoader(PDF_PATH).load()
chunks = RecursiveCharacterTextSplitter(
    chunk_size=1000, chunk_overlap=300
).split_documents(docs)
# Result: 116 pages -> 461 chunks
```

PyPDFLoader.load() returns one LangChain Document per PDF page — 116 of them, each carrying the page's raw text plus metadata (crucially, the page number, which is what later lets the evaluation check “did we retrieve the correct page”).

RecursiveCharacterTextSplitter then breaks each page into smaller pieces. “Recursive” is the important word here — it does not just cut every 1000 characters blindly. It tries a prioritized list of separators (paragraph break → line break → space → character) and only falls back to a harder split when a chunk is still too big. This means it tries hard to keep a paragraph or sentence intact rather than slicing through the middle of one.

The math of chunk_size=1000, chunk_overlap=300

chunk_size=1000 characters (roughly 150–200 English words, since average word+space length is ~5–6 characters) sets the target chunk length. chunk_overlap=300 means each new chunk repeats the last 300 characters of the previous chunk before continuing.

Why overlap at all? Picture a sentence that states “the MPC cut the repo rate by 25 basis points to 6.25%” sitting exactly on a chunk boundary — without overlap, chunk A might end with “...cut the repo rate by 25 basis” and chunk B might start with “points to 6.25%.” Neither chunk alone contains the full fact, so neither would be a useful retrieval hit. A 300-character overlap (30% of the chunk) makes it very likely the full sentence survives intact inside at least one chunk.

The observed ratio — 116 pages → 461 chunks, about 4 chunks per page — implies each page holds roughly 3,000–4,500 characters of extractable text, consistent with a dense, small-font policy report page.

Why chunk instead of embedding whole pages or the whole document: the embedding model has a maximum input length (MiniLM effectively attends well up to ~256 tokens), and even within that limit, embedding a long passage produces a single vector that is an average of many different ideas — diluting the vector so it doesn't match any specific query well. Smaller, focused chunks produce sharper, more specific embeddings.

2.3 Embeddings — sentence-transformers/all-MiniLM-L6-v2

```
embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)
vectordb = Chroma.from_documents(chunks, embeddings)
```

An embedding model is a function that maps a piece of text to a fixed-length vector of numbers (here, 384 numbers) such that texts with similar meaning end up as vectors that point in similar directions in that 384-dimensional space. “Repo rate cut” and “policy rate reduced” will land near each other even though they share almost no words in common — this is the entire reason embeddings beat plain keyword search for this task.

What MiniLM actually is (model architecture intuition)

MiniLM is a distilled Transformer: a smaller student network (6 transformer layers, hence “L6”) trained to mimic the behavior of a much larger teacher model (e.g. BERT-base, 12 layers), through a process called knowledge distillation. The student is trained to match the teacher's internal attention patterns and output distributions rather

than being trained from scratch — this lets it retain most of the teacher's quality at a fraction of the size and inference cost.

On top of distillation, this specific checkpoint (all-MiniLM-L6-v2) was further fine-tuned using a Sentence-BERT-style contrastive objective on over a billion sentence pairs, specifically so that mean-pooling the token embeddings produces a good whole-sentence representation. This distinction matters: a raw BERT model's embeddings are not naturally good for similarity search out of the box — it takes this extra sentence-level contrastive training step (the core insight of the original Sentence-BERT paper, Reimers & Gurevych, 2019) to make cosine similarity between two sentence embeddings actually mean something.

The math: cosine similarity, with a worked example

Chroma compares the query's embedding vector to every stored chunk vector using cosine similarity:

$$cos(\theta) = (A \cdot B) / (\|A\| \times \|B\|)$$

Where A·B is the dot product (sum of element-wise multiplication) and ‖A‖ is the vector's magnitude (square root of the sum of squares). Dividing by the magnitudes removes the effect of vector length, so cosine similarity purely measures the angle — i.e. direction/meaning — between two vectors, regardless of how “long” the text was.

Worked toy example (using 3 dimensions instead of 384, for hand-calculation): let A = [1, 2, 0] represent one chunk and B = [2, 1, 0] represent a query.

- Dot product: A·B = (1×2) + (2×1) + (0×0) = 2 + 2 + 0 = 4
- ‖A‖ = √(1²+2²+0²) = √5 ≈ 2.236
- ‖B‖ = √(2²+1²+0²) = √5 ≈ 2.236
- cos(θ) = 4 / (2.236 × 2.236) = 4 / 5 = 0.8

A cosine similarity of 0.8 (out of a max of 1.0) means the two vectors point in a fairly similar direction — in embedding space, that translates to “these two pieces of text are semantically close.” In the real system this is the same calculation, just across 384 dimensions instead of 3, run between the query vector and all 461 stored chunk vectors, returning the highest-scoring ones.

Why MiniLM (384-dim, free, local) instead of a bigger embedding model like OpenAI's text-embedding-3 (1536-dim, paid API): MiniLM is fast and free to run locally, which matters for an evaluation loop that re-embeds queries dozens of times. The trade-off is quality — a smaller embedding space and less training data means MiniLM is more likely to conflate subtly different financial concepts (e.g. “repo rate” vs “reverse repo rate” vs “MSF rate”) than a larger, domain-richer embedding model would. This is a legitimate, defensible limitation to name proactively in an interview.

2.4 The vector store — ChromaDB

Chroma stores every chunk's embedding vector alongside its original text and metadata (including the page number), and builds an index that supports fast approximate nearest-neighbor (ANN) search. Under the hood, vector databases like Chroma typically use a graph-based index called HNSW (Hierarchical Navigable Small World) — rather than comparing the query to all 461 vectors one by one (a full brute-force scan), HNSW organizes vectors into layered graphs so the search can “jump” through a sparse top layer to get close to the answer quickly, then refine through denser lower layers. This turns an O(N) brute-force search into an approximate O(log N) search — the benefit is negligible at 461 chunks, but the same code would scale to millions of chunks without a rewrite, which is the actual point of using a real vector DB instead of, say, a plain NumPy array.

Why Chroma specifically (over Pinecone, Weaviate, FAISS): Chroma runs fully in-process with zero setup, zero cost, and no network calls — ideal for a Colab notebook and rapid iteration. In a production system serving real users you'd likely reach for a managed vector DB for durability, scaling, and multi-user access, which is worth saying out loud as “next step” thinking.

2.5 Retriever + LLM — building the RAG chain

```
llm = ChatGroq(model="llama-3.1-8b-instant", temperature=0)
retriever = vectordb.as_retriever(search_kwargs={"k": 4})
prompt = ChatPromptTemplate.from_template(
    "Use ONLY this context to answer. If not found, say so.\n\n"
    "Context:\n{context}\n\nQuestion: {question}\n\nAnswer:"
)

def ask(question):
    docs = retriever.invoke(question)
    context = "\n\n".join(d.page_content for d in docs)
    messages = prompt.format_messages(context=context, question=question)
    response = llm.invoke(messages)
    return response.content, docs
```

This is the core RAG loop: retrieve the top-4 chunks by cosine similarity, concatenate their text into a single context block, drop that context and the question into the prompt template, and send it to the LLM.

Why temperature=0: temperature controls how the LLM samples its next token — at temperature 0, it always picks the single highest-probability token (greedy decoding), producing deterministic, repeatable output. For a factual finance Q&A tool this is exactly what you want: the same question should get the same answer every time, and you want the model favoring the most statistically likely (and hopefully most grounded) continuation rather than a more “creative”, higher-variance one.

Why Llama-3.1-8b-instant via Groq: Groq's custom inference hardware (LPUs) makes even API calls extremely fast and, on the free tier, effectively costless — important because the evaluation loop later calls the LLM dozens of times. The real trade-off: 8 billion parameters is small by current standards (vs. 100B+ class models like GPT-4 or Claude). Smaller models are more likely to misread nuanced context or state a number slightly wrong, which is very plausibly why the DeepEval Correctness score came out at only 0.58 despite Contextual Recall being a much healthier 0.78 — the right information was retrieved, but the small model didn't always use it correctly. Being able to say this diagnosis out loud is one of the strongest things you can demonstrate in an interview.

Why the prompt explicitly says “if not found, say so”: this is the single most important anti-hallucination instruction in the whole system. Without it, an LLM under-specified by weak context will often still confidently invent a plausible-sounding number rather than admit uncertainty — which is the worst possible failure mode for a finance tool.

2.6 Two-stage retrieval — adding a cross-encoder reranker

```
reranker = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")

def retrieve_and_rerank(query, initial_k=15, final_k=4):
    candidates = vectordb.similarity_search(query, k=initial_k)
    pairs = [[query, doc.page_content] for doc in candidates]
    scores = reranker.predict(pairs)
    ranked = sorted(zip(candidates, scores),
                    key=lambda x: x[1], reverse=True)
    return [doc for doc, score in ranked[:final_k]]
```

Bi-encoder vs. cross-encoder — the key distinction to be able to explain

The MiniLM embedding model used for initial retrieval is a bi-encoder: it encodes the query and each document completely independently into separate vectors, and only compares them afterward via cosine similarity. This is fast and scales to millions of documents because every document's vector can be pre-computed once, offline, and stored — at query time you only need to embed the query itself and do a vector comparison.

A cross-encoder works differently: it feeds the query and a candidate document into the transformer together, as a single input pair, so the model's attention layers can directly compare query words against document words token-

by-token before producing one relevance score. This joint attention makes cross-encoders meaningfully more accurate at judging true relevance — but it is far more expensive, because the full transformer has to be re-run for every single query-document pair, with no possibility of pre-computing anything offline.

This is exactly why the pipeline uses both, in two stages: the cheap bi-encoder first narrows 461 chunks down to a manageable 15 candidates (initial_k=15), and only then does the expensive cross-encoder carefully re-score those 15 to pick the best 4 (final_k=4). Running the cross-encoder over all 461 chunks for every query would work just as well accuracy-wise but would be far too slow for real use — this two-stage "retrieve-then-rerank" pattern is standard architecture in production search and RAG systems.

Why the reranker model is trained on MS MARCO: MS MARCO is a large public dataset of real Bing search queries paired with human-judged relevant passages. Training on it teaches the cross-encoder specifically the skill this pipeline needs — "given a question and a candidate passage, how relevant is this passage to answering it" — which is a different skill than the general-purpose semantic similarity the embedding model was trained for.

Why MRR improved (0.564 → 0.758) but Hit-Rate@4 stayed flat at 80%: this is the single best insight to lead with in an interview about this project. Reranking can only reorder candidates that the bi-encoder already retrieved into its initial_k=15 pool — it cannot pull in a page the first-stage retrieval missed entirely. So when the correct page was somewhere in that top-15, the reranker got much better at pushing it up to rank 1–2 (raising MRR sharply). But on the handful of queries where the bi-encoder's initial retrieval didn't surface the correct page in its top-15 at all, no amount of reranking can fix that — which is why Hit-Rate@4, a pure recall metric, was untouched by adding the reranker. In short: reranking improves ranking quality, not retrieval coverage.

2.7 The evaluation set

30 question/answer pairs were hand-written directly against the PDF, each one tagged with its ground_truth_page — the exact page number in the report where the correct answer actually appears. Topics deliberately span the whole document: GDP growth, CRR/repo rate decisions, inflation (headline, food, core), liquidity conditions, government borrowing, external/global comparisons — rather than clustering questions in one section, which would make retrieval look artificially easier or harder than it really is.

Why page-level ground truth instead of just an expected answer string: it decouples two very different failure modes that are otherwise easy to conflate: did the retriever fail to find the right source material (a retrieval problem), or did the retriever find it but the LLM still answered wrong (a generation problem)? Tagging each question with the exact correct page lets Hit-Rate@k and MRR measure retrieval in complete isolation from the LLM.

2.8 Retrieval metrics — Hit-Rate@k and MRR, with worked examples

```
def retrieval_metrics(retriever, eval_df, k=4):
    hits = 0
    reciprocal_ranks = []
    for _, row in eval_df.iterrows():
        docs = retriever.invoke(row["question"])
        pages = [d.metadata.get("page") + 1 for d in docs]
        gt_page = int(row["ground_truth_page"])
        if gt_page in pages:
            hits += 1
            rank = pages.index(gt_page) + 1
            reciprocal_ranks.append(1.0 / rank)
        else:
            reciprocal_ranks.append(0.0)
    hit_rate = hits / len(eval_df)
    mrr = sum(reciprocal_ranks) / len(reciprocal_ranks)
    return hit_rate, mrr
```

Hit-Rate@k

Hit-Rate@k = (number of queries where the correct page appears anywhere in the top-k results) / (total number of queries)

It's a simple yes/no per query — it doesn't care whether the correct page came back at rank 1 or rank 4, only whether it showed up at all within the top k.

Mean Reciprocal Rank (MRR)

$MRR = (1 / |Q|) \times \Sigma\ (1 / rank_i)$

For each query, the "reciprocal rank" is 1 divided by the position at which the correct result first appears (1 if it's the very first result, 1/2 if second, 1/3 if third, and 0 if it never appears at all). MRR is simply the average of that value across all queries.

Worked example with 3 queries:

Query	Correct page found at rank	Reciprocal Rank
Q1	1	1 / 1 = 1.000
Q2	3	1 / 3 = 0.333
Q3	not found in top-k	0

MRR = (1.000 + 0.333 + 0.000) / 3 = 0.444

Why MRR matters more than Hit-Rate alone: Hit-Rate treats a correct result at rank 1 and rank 4 identically — both just count as "a hit." But in the actual generation step, all top-k chunks get concatenated into one context block and handed to the LLM together; a correct chunk buried last is more likely to be under-weighted or overlooked by the model than one placed first. MRR is sensitive to that ordering in a way Hit-Rate is blind to, which is exactly why reranking (which only reorders, never changes what's included) could improve MRR from 0.564 to 0.758 while Hit-Rate@4 held flat at 80%.

2.9 DeepEval — LLM-as-judge generation evaluation

```
class GroqJudge(DeepEvalBaseLLM):
    def __init__(self, model_name="llama-3.1-8b-instant"):
        self.model = ChatGroq(model=model_name, temperature=0)
    def generate(self, prompt: str) -> str:
        return self.model.invoke(prompt).content
    ...

correctness = GEval(
    name="Correctness",
    criteria="Determine if the actual output is factually
              consistent with the expected output.",
    evaluation_params=[ACTUAL_OUTPUT, EXPECTED_OUTPUT],
    model=judge, threshold=0.5,
)
relevancy = ContextualRelevancyMetric(model=judge, threshold=0.5)
recall = ContextualRecallMetric(model=judge, threshold=0.5)
```

DeepEval's core idea is to use a second LLM as an automatic grader ("LLM-as-judge"), instead of exact string matching, because two financially-correct answers can be worded completely differently ("the repo rate was cut by 25 bps" vs. "the MPC reduced the policy rate by 0.25 percentage points"). A word-overlap metric like ROUGE or BLEU would score these as very different even though they're both correct — an LLM judge can recognize they mean the same thing.

Because Groq's API doesn't expose token-level log-probabilities the way OpenAI's does, this project wraps ChatGroq in a custom GroqJudge class implementing DeepEval's expected DeepEvalBaseLLM interface — this is a

necessary plumbing step to let a free, open-weight model serve as the judge instead of defaulting to a paid OpenAI judge model.

GEval / Correctness

GEval implements the G-Eval method (Liu et al., 2023): rather than asking the judge model for a bare number, it has the judge produce its own chain-of-thought evaluation steps for the given criteria, then score the actual output against the expected output using a structured "form-filling" approach. (The original G-Eval paper computes a continuous score by weighting each discrete score option, e.g. 1–5, by its output token probability; implementations without access to log-probabilities, as here, approximate this with a direct scored judgment instead.) Average Correctness here was 0.58 — middling, and consistent with the earlier diagnosis that the small 8B generator model doesn't always state retrieved facts precisely.

Contextual Relevancy

This metric asks: of everything in the retrieved context that was handed to the LLM, what fraction was actually relevant to the question? Mechanically, the judge LLM extracts individual statements from the retrieved chunks and classifies each as relevant or irrelevant to the specific query, then relevancy = (relevant statements) / (total statements). Average Contextual Relevancy here was only 0.28 — a clear, honest signal that a lot of retrieved text is tangential padding rather than directly on-topic, most likely a side effect of chunk_size=1000 pulling in surrounding paragraphs beyond just the answer-bearing sentence.

Contextual Recall

This metric asks the reverse question: does the retrieved context contain enough information to support everything in the expected/ground-truth answer? The judge LLM breaks the expected answer into individual claims and checks whether each one can be traced back to something in the retrieved context; recall = (supported claims) / (total claims in the expected answer). Average Contextual Recall here was a healthy 0.78 — meaning the retrieval pipeline usually does surface the needed facts, even though (per the relevancy score) it also brings along a fair amount of noise, and even though (per the correctness score) the generator doesn't always use those facts perfectly.

The single most important diagnostic sentence to have ready: "Contextual Recall (0.78) is meaningfully higher than Correctness (0.58), which tells me the bottleneck in this system is generation, not retrieval — the right information is usually being found, but the small 8B model doesn't always state it correctly. If I were improving this next, I'd upgrade the generator model before I'd touch the retriever." This one sentence demonstrates you understand what the numbers mean, not just that you ran the code.

2.10 Retry logic and checkpointing

```
def measure_with_retry(metric, tc, max_retries=5):
    for attempt in range(max_retries):
        try:
            metric.measure(tc)
            return True
        except Exception as e:
            if "rate_limit" in str(e).lower():
                wait = 20 * (attempt + 1)
                time.sleep(wait)
            ...
```

Free-tier Groq API calls are rate-limited, and the evaluation loop makes a large number of judge calls (30 questions × 3 metrics, each potentially multiple LLM calls internally). measure_with_retry() catches rate-limit errors specifically and backs off for an increasing wait (20s, 40s, 60s...) before retrying, rather than crashing the whole run on the first 429 error.

Results are also written to a checkpoint CSV after every single test case, and the script checks for and resumes from that checkpoint on restart. This is ordinary production engineering hygiene applied to a research script — a 30-question × 3-metric evaluation with multi-second LLM calls and retry waits can easily run for 15–30 minutes, and

losing that progress to a disconnected Colab session would be a real cost. Mentioning this unprompted in an interview signals you think about reliability, not just model accuracy.

3. Final Results, All in One Place

Metric	Score	What it measures
Hit-Rate@4 (base retrieval)	80%	Correct page found anywhere in top-4
MRR (base retrieval)	0.564	How highly-ranked the correct page was
MRR (after reranking)	0.758	Same, after cross-encoder reordering
Avg. Correctness (DeepEval)	0.58	Is the final answer factually right
Avg. Contextual Relevancy	0.28	How much retrieved text was on-topic
Avg. Contextual Recall	0.78	Did retrieved text cover the expected answer

4. Likely Interview Questions and How to Answer Them

Q: Why did you choose a bi-encoder + cross-encoder two-stage retrieval instead of just using the cross-encoder on everything?

A: Cross-encoders require running the full transformer jointly on every query-document pair, so scoring all 461 chunks against every query would be far too slow for interactive use. The bi-encoder cheaply narrows the field to 15 candidates using pre-computed vectors, and the cross-encoder only has to do its expensive, more accurate scoring on those 15 — this retrieve-then-rerank pattern is standard in production search systems.

Q: Why did MRR improve with reranking but Hit-Rate@4 didn't?

A: Reranking only reorders candidates already inside the initial top-15 pool from the bi-encoder — it can't recover a page the first-stage retrieval missed entirely. So it improved ranking position for correct pages that were already being found (raising MRR), but couldn't fix the handful of queries where the correct page wasn't retrieved at all in the first place (so Hit-Rate@4 stayed at 80%).

Q: Your Contextual Relevancy score (0.28) is quite low — why, and how would you fix it?

A: With chunk_size=1000 and overlap=300, each retrieved chunk likely contains surrounding paragraphs beyond just the sentence that answers the question, so a lot of retrieved text is topically adjacent but not directly relevant. I'd address this by shrinking chunk size, adding sentence-level re-splitting on top of the current recursive splitter, or increasing the reranker's selectivity so only the most tightly relevant chunks survive to the final context.

Q: Contextual Recall (0.78) is much higher than Correctness (0.58) — what does that tell you?

A: It tells me the bottleneck is generation, not retrieval. The system usually does find the needed facts, but the small 8B Llama model doesn't always state them precisely in the final answer. The next improvement I'd make is upgrading the generator model, not the retriever.

Q: Why temperature=0 for the LLM?

A: Temperature controls sampling randomness; at 0 the model always greedily picks the highest-probability next token, giving deterministic, repeatable output. For a factual finance Q&A system I want the same question to reliably produce the same answer, and I want the model favoring the statistically safest continuation rather than a more ‘creative’ one.

Q: Why MiniLM for embeddings instead of a bigger commercial embedding model?

A: MiniLM is free, runs locally, and is fast enough to support an evaluation loop that embeds many queries. The trade-off is representational quality — a smaller model with a smaller embedding space is more likely to conflate related-but-distinct financial terms. In a production system I'd benchmark a larger embedding model to see if it closes some of the Contextual Relevancy gap.

Q: What's the difference between Hit-Rate@k and MRR, and why report both?

A: Hit-Rate@k is a binary yes/no per query — did the correct result appear anywhere in the top k. MRR additionally rewards how highly it was ranked. Reporting both lets you separate two different failure modes: Hit-Rate tells you about raw retrieval coverage, MRR tells you about ranking quality within that coverage — which is exactly how I diagnosed that reranking fixed ranking but not coverage.

Q: Why use DeepEval / LLM-as-judge instead of just comparing strings?

A: Financially correct answers can be phrased many different ways, so exact-match or word-overlap metrics like ROUGE would falsely penalize a correct answer that's worded differently from the expected one. An LLM judge can assess whether two answers are factually equivalent regardless of exact phrasing.

Q: What would you change if you rebuilt this project today?

A: Three things: shrink the chunk size (or add sentence-level splitting) to raise Contextual Relevancy; swap the 8B generator for a larger model to close the Correctness gap given Recall is already healthy; and add metadata filtering (e.g. by report section) since right now retrieval treats the whole 116-page document as one undifferentiated pool.